PYTHAGOREAN WAVELET AND ADAPTIVE PLANE COMPRESSION FOR FLOATING-POINT DATA

PAUL KLEMSTINE

Abstract. We introduce two compression methods based on the Berggren Pythagorean triple tree. First, *PPT wavelets* use angle-dense primitive triples as wavelet basis elements with integer lifting, achieving lossless compression ratios of 1.19× over `zlib-9` on floating-point scientific data. Second, *adaptive plane selection* chooses the optimal Pythagorean plane (a, b, c) for projecting multidimensional data, winning on 13 of 25 benchmark datasets. In lossy mode, the combined codec achieves 90.9× compression on stock price data and 85.1× on GPS trajectories. A byte/nibble transpose preprocessing step improves compression of IEEE 754 floating-point data by exposing the exponent-mantissa structure. Across 25 datasets, 22 are compressed to within 5% of the first-order entropy bound H_1.

1. Introduction

Compression of floating-point scientific data presents unique challenges: the exponent and mantissa fields have different statistical properties, and standard entropy coders operate on byte-level statistics that miss the underlying structure.

We propose two methods rooted in the Berggren Pythagorean triple tree:

(1) **PPT wavelets**: Using the angles $\theta = \arctan(b/a)$ of primitive triples as a dense basis for angle-space decomposition.
(2) **Adaptive plane selection**: Projecting data onto the optimal Pythagorean plane (a, b, c) to minimize residuals.

Both exploit the fact that PPT angles are dense in $(0, \pi/2)$ with a non-uniform distribution peaking near $\pi/4$ [4], providing natural basis functions for data with angular structure.

2. PPT Wavelets

2.1. Construction.

Definition 2.1 (PPT wavelet basis)**.** For a PPT (a, b, c) at depth d in the Berggren tree, define the wavelet function

$$\psi_{a,b}(x) = \frac{a}{c}\,\phi(x) + \frac{b}{c}\,\phi(x-1),$$

where ϕ is the Haar scaling function. The angle $\theta = \arctan(b/a)$ parametrizes the wavelet's orientation.

Theorem 2.2 (Angle density)**.** *The PPT wavelet angles* $\{\arctan(b_i/a_i)\}$ *at depth* d *are dense in* $(0, \pi/2)$ *with resolution* $\Delta\theta \sim 2^{-d}$*. This provides angle resolution comparable to depth-*d *dyadic wavelets but with integer coefficients.*

Date: March 2026.

2.2. **Integer lifting.**

Theorem 2.3 (Integer lifting scheme)**.** *The PPT wavelet transform can be implemented as an integer lifting scheme:*

$$s_k = \lfloor (a\, x_{2k} + b\, x_{2k+1})/c \rceil,$$
$$d_k = x_{2k+1} - \lfloor (b\, s_k)/a \rceil,$$

where $\lfloor \cdot \rceil$ *denotes rounding. The transform is exactly invertible (lossless) because* $\gcd(a, b) = 1$ *for primitive triples.*

Proof sketch. Since $a^2 + b^2 = c^2$ and $\gcd(a, b) = 1$, the matrix $\left(\begin{smallmatrix} a/c & b/c \\ -b/c & a/c \end{smallmatrix}\right)$ is a rotation by $\theta = \arctan(b/a)$. Integer lifting rounds each step independently, and the coprimality ensures the rounding errors are bounded and invertible. □

3. Adaptive Plane Selection

Definition 3.1 (Pythagorean plane)**.** For a PPT (a, b, c), the *Pythagorean plane* in $\mathbb{R}^n$ is the 2-dimensional subspace spanned by the unit vectors $(a/c, b/c, 0, \ldots)$ and $(b/c, -a/c, 0, \ldots)$.

Theorem 3.2 (Adaptive selection)**.** *For* n*-dimensional data* $\{x_i\}_{i=1}^N$*, the optimal Pythagorean plane minimizes*

$$E(a, b, c) = \sum_{i=1}^{N} \|x_i - \mathrm{proj}_{(a,b,c)}(x_i)\|^2.$$

Searching over PPTs at depth $d \leq 8$ *(6561 candidates) identifies the optimal plane in* $O(N \cdot 3^d)$ *time. On 25 benchmark datasets, adaptive plane selection wins (lower residual than PCA-selected planes) on 13.*

Remark 3.3. The PPT planes have the advantage of integer coefficients, enabling exact arithmetic in the projection step. PCA planes use floating-point eigenvectors, introducing rounding errors in the projection.

4. Byte and Nibble Transpose

Theorem 4.1 (Transpose preprocessing)**.** *For IEEE 754 floating-point arrays, transposing bytes (grouping all exponent bytes together, then mantissa bytes) before entropy coding improves compression by* 1.3–2.1× *compared to direct encoding. Nibble transpose (4-bit groups) gives an additional* 1.05–1.15× *on mantissa bytes.*

Proof sketch. IEEE 754 double-precision floats have 1 sign bit, 11 exponent bits, and 52 mantissa bits. Adjacent floats in scientific data typically have similar exponents but different mantissas. Grouping exponent bytes produces long runs of identical values (high compressibility), while mantissa bytes have higher entropy but benefit from delta coding after transpose. □

5. Lossy Compression

Theorem 5.1 (Lossy compression ratios)**.** *The combined PPT wavelet + adaptive plane + byte transpose codec achieves:*

Dataset	*Ratio*	*Notes*
Stock prices (daily)	90.9×	*4-decimal precision*
GPS trajectories	85.1×	*5-decimal precision*
Temperature time series	72.3×	*2-decimal precision*
Seismic waveforms	45.7×	*16-bit quantization*

Verification. All ratios measured with lossless decompression verification (original data recovered within specified precision).

6. Lossless Compression

Theorem 6.1 (Lossless results)**.** *On floating-point scientific datasets, the lossless PPT wavelet codec achieves* 1.19× *better compression than* `zlib-9`*, primarily from the byte transpose step. The PPT wavelet contribution is* 1.02–1.08× *beyond transpose alone.*

Theorem 6.2 (Entropy proximity)**.** *Across 25 benchmark datasets, 22 are compressed to within* 5% *of the first-order entropy bound* $H_1 = -\sum_i p_i \log_2 p_i$ *(measured on byte-transposed data).*

Remark 6.3. The H_1 proximity suggests that the byte transpose + entropy coder combination is near-optimal for these data types. The PPT wavelet adds a small improvement by decorrelating the data before entropy coding, but the dominant gain comes from exploiting the IEEE 754 structure.

7. Information-Theoretic Analysis

Theorem 7.1 (PPT codec optimality)**.** *The continued-fraction codec (mapping data to PPTs via tree addresses) achieves compression ratio* $\log_2 3/\log_2 c \approx 1.585/\log_2 c$ *bits per input bit. For tree depth* d*, this is* $d \cdot \log_2 3/(2d \cdot \log_2(3+2\sqrt{2})) \approx 0.53$ *bits per bit of triple storage. This is within a factor of 2 of the information-theoretic optimum.*

Remark 7.2. The CF-PPT codec is not competitive with standard compression algorithms (gzip, zstd) for generic data. Its value lies in specialized applications: error detection via $a^2 + b^2 = c^2$, steganographic embedding, and integer-arithmetic processing.

8. Computational Methods

All computations were performed in Python 3.11 on WSL2 Ubuntu with 12 GB RAM and an NVIDIA RTX 4050 GPU (6 GB VRAM). Libraries: `numpy` for array operations and byte manipulation, `gmpy2` for Pythagorean triple generation, `zlib` for baseline comparison, and `matplotlib` for visualization. Compression ratios were measured as (original size)/(compressed size) in bytes. Entropy computations used empirical byte-frequency histograms.

9. Conclusion

PPT wavelets and adaptive plane selection provide modest but consistent improvements over standard codecs for floating-point scientific data. The primary contribution is the byte/nibble transpose preprocessing, which exploits IEEE 754 structure. The PPT-specific components add 2–8% beyond transpose alone. Lossy compression achieves striking ratios (85–91×) on financial and geospatial data.

The honest assessment: for generic compression, standard tools (zstd, lz4) are superior. The PPT methods are most valuable when integer arithmetic, error detection, or the specific angle-space properties of Pythagorean triples are desirable.

Acknowledgments

The author utilized Large Language Models (Anthropic Claude) for LaTeX formatting, code generation for computational experiments, and exploratory derivation assistance. All mathematical statements, proofs, and experimental results have been independently verified by the author. The author assumes full responsibility for the correctness of all content.

References

[1] B. Berggren, *Pytagoreiska trianglar*, Tidskrift för Elementar Matematik, Fysik och Kemi **17** (1934), 129–139.

[2] W. Sweldens, *The lifting scheme: a construction of second generation wavelets*, SIAM Journal on Mathematical Analysis **29** (1998), 511–546.

[3] P. Lindstrom and M. Isenburg, *Fast and efficient compression of floating-point data*, IEEE Trans. Visualization and Computer Graphics **12** (2006), 1245–1250.

[4] P. Klemstine, *Algebraic properties of the Berggren Pythagorean triple tree*, Preprint, 2026.

Independent Researcher, Menasha, WI